Best Case, Worst Case & Average Case Analysis

- Best case is the function which performs the minimum number of steps on input data of n elements.
- Worst case is the function which performs the maximum number of steps on input data of size n.
- Average case is the function which performs an average number of steps on input data of n elements.

- In bubble sort, when the input array is already sorted, the time taken by the algorithm is linear i.e. the best case.
- But, when the input array is in reverse condition, the algorithm takes the maximum time (quadratic) to sort the elements i.e. the worst case.
- When the input array is neither sorted nor in reverse order, then it takes average time.
- These durations are denoted using asymptotic notations.

Asymptotic Notation

- Asymptotic notations are the mathematical notations used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n.

- There are three different notations:
    1. big O,
    2. big Theta (Θ),
    3. big Omega (Ω)

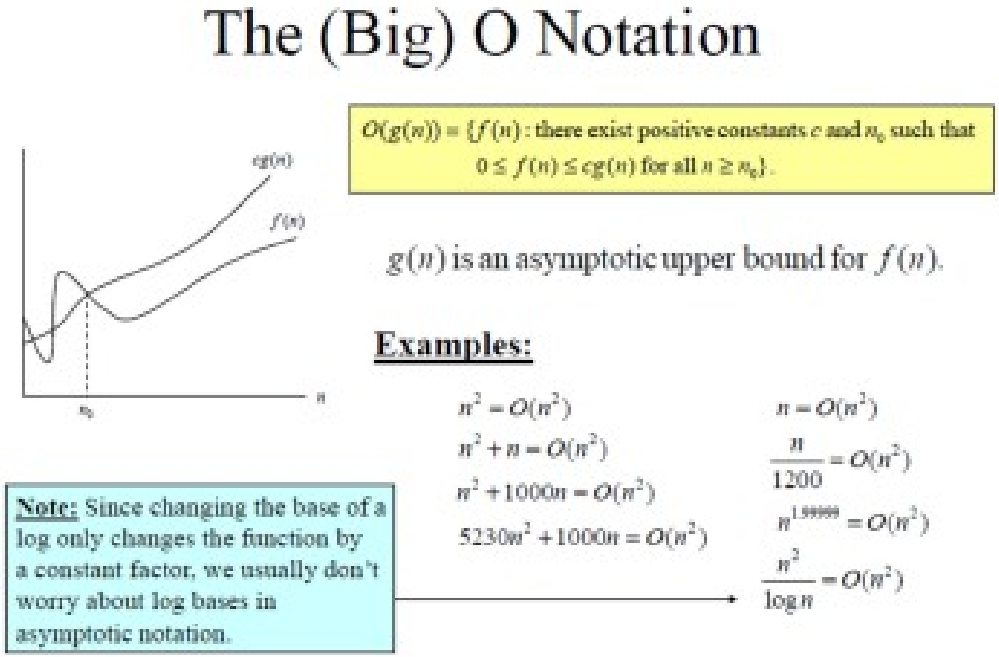
- big-Θ is used when the running time is the same for all cases,

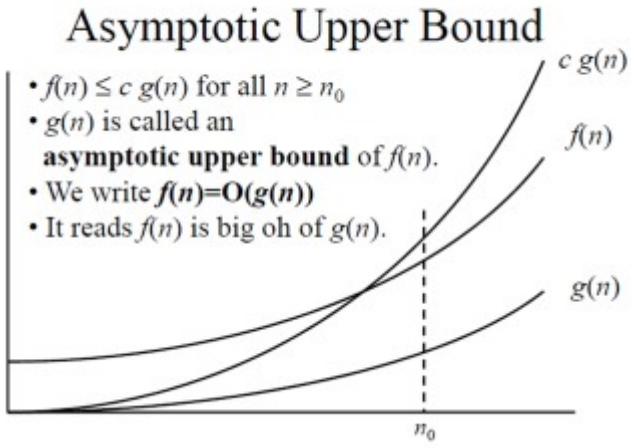
- big-O for the worst case running time,
- big-Ω for the best case running time.

Big-O Notation

- The Big-O notation describes the worst-case running time of a program.
- We compute the Big-O of an algorithm by counting how many iterations an algorithm will take in the worst-case scenario with an input of N.
- Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.
- We typically consult the Big-O because we must always plan for the worst case.

  O(g(n)) ={f(n): there exist positive constants c and n_0 such that $0 \le f(n) \le cg(n)$ for all $n \ge n_0$}





- The above expression can be described as a function f(n) belongs to the set O(g(n)) if there exists a positive constant c such that it lies between 0 and cg(n), for sufficiently large n.
- For any value of n, the running time of an algorithm does not cross the time provided by O(g(n)).
- Since it gives the worst-case running time of an algorithm, it is widely used to analyze an algorithm as we are always interested in the worst-case scenario.

Omega Notation (Ω-notation)

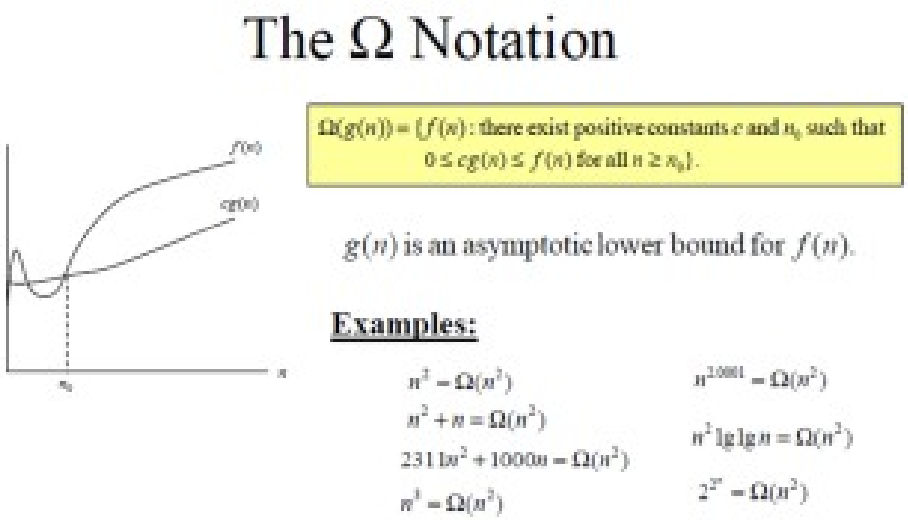
Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

$\Omega(g(n)) = \{ f(n)$: there exist positive constants c and n_0
such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0 \}$

The above expression can be described as a function f(n) belongs to the set Ω(g(n)) if there exists a positive constant c such that it lies above cg(n), for sufficiently large n.

For any value of n, the minimum time required by the algorithm is given by

Omega Ω(g(n))



Theta Notation (Θ-notation)

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm

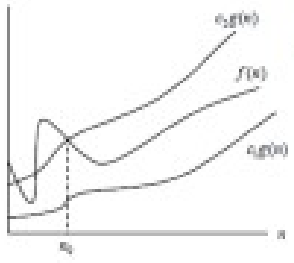
For a function g(n), Θ(g(n)) is given by the relation:

Θ(g(n)) = { f(n): there exist positive constants c_1, c_2 and n_0
such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0$ }

The above expression can be described as a function f(n) belongs to the set Θ(g(n)) if there exist positive constants c_1 and c_2 such that it can be sandwiched between $c_1 g(n)$ and $c_2 g(n)$, for sufficiently large n.

If a function f(n) lies anywhere in between $c_1 g(n)$ and $c_2 g(n)$ for all $n \ge n0$, then f(n) is said to be asymptotically tight bound.

The Θ Notation



$\Theta(g(n)) = \{f(n)$: there exist positive constants c_1, c_2, and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0\}$.

$g(n)$ is an asymptotically tight bound for $f(n)$.

Example:

$\frac{n^2}{2} - 2n = \Theta(n^2)$, with $c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$, and $n_0 = 8$.